

Dokumentacja techniczna projektu Discord Bot — TV, Giełda & Asystent AI

Architektura aplikacji · biblioteki Python · Docker · CI/CD (GitHub Actions)

Materiał przygotowawczy do rozmowy kwalifikacyjnej

Repozytorium: `tv_and_stockmarket_bot`

Stack: Python 3.12 · discord.py · Flask · SQLite · Docker · GitHub Actions · Oracle Cloud

1. Synopsis — 30-sekundowa prezentacja projektu

Poniższa wypowiedź to „elevator pitch” — warto znać jej kształt na pamięć. Zawiera język, framework, architekturę, sposób przechowywania danych, integracje zewnętrzne oraz pełną ścieżkę wdrożenia.

„To wielofunkcyjny bot na Discorda napisany w Pythonie z użyciem **discord.py**. Jeden proces łączy kilkanaście obszarów funkcjonalnych: śledzenie premier seriali i filmów, notowania giełdowe i monitoring portfela, pogodę, przypomnienia, nawyki, listę książek i gier oraz asystenta AI opartego o Google Gemini. Pobiera dane z API (TMDB, Yahoo Finance, OpenWeatherMap, Gemini) i zapisuje wszystko w SQLite. Kod jest podzielony na **cogi** — po jednym na funkcję — oraz warstwową warstwę dostępu do danych, dzięki czemu każdy obszar jest odizolowany i testowalny. Aplikacja jest skonteneryzowana (Docker) i wdrażana przez pipeline CI/CD w GitHub Actions: każdy push do **main** uruchamia testy, buduje obraz, wypycha go do GitHub Container Registry, a następnie loguje się przez SSH na darmową maszynę Oracle Cloud i przewdraża kontener. Sekrety produkcyjne żyją w pliku **.env** na serwerze i nigdy nie trafiają do repozytorium.”

2. Opis aplikacji — architektura

Aplikacja to **bot na Discorda** — długo działający proces Pythona, który łączy się z bramą (gateway) Discorda po WebSocket i reaguje na dwa rodzaje komend:

- **Komendy z prefiksem** (`!help`, `!sync`) — starszy styl.
- **Komendy slash** (`/weather`, `/ping`) — nowoczesne „application commands” Discorda.
- Wiele komend to **komendy hybrydowe** (`bot.hybrid_command`) — ta sama funkcja dostępna jest na oba sposoby.

Trzy warstwy architektury

1. Punkt wejścia — `bot.py`

- Najpierw konfiguruje logowanie (`logger.setup_logging()`) — kolejność ma znaczenie, by nic nie logowało się bez formatowania.
- Ładuje konfigurację, definiuje **intents** (`message_content = True`) i tworzy instancję `commands.Bot`.
- Ładuje ~13 **cogów** z listy `INITIAL_EXTENSIONS`.
- Posiada globalny **handler błędów** komend slash (`@bot.tree.error`) — jedna wadliwa komenda nigdy nie wyrzuci bota.
- W zdarzeniu `on_ready` **synchronizuje drzewo komend** z Discordem (na poziomie serwera — natychmiast, oraz globalnie — do godziny propagacji).

2. Cogi — katalog `cogs/` (moduły funkcji)

Cogi to system wtyczek w discord.py. Każdy plik (`stocks.py`, `tv_shows.py`, `gemini.py`, `reminders.py`, ...) to samodzielna klasa grupująca powiązane komendy i pętle w tle. Dzięki temu duży bot pozostaje łatwy w utrzymaniu — każda funkcja jest odizolowana, można ją ładować/odładować niezależnie, a awaria jednego coga nie zatrzymuje pozostałych.

3. Warstwa danych — `data_manager.py` + `data_manager_impl/`

`data_manager.py` to cienka **fasada**: właściwa implementacja jest podzielona na **miksiny** (`StocksMixin`, `RemindersMixin`, `MediaMixin`, `ProductivityMixin` ...), które przez wielodziedziczenie składają się w jedną klasę `DataManager`. Publiczne API pozostaje `from data_manager import DataManager`, więc można refaktoryzować wewnątrz bez dotykania kodu wywołującego.

- Trwałość oparta o **SQLite** (`data/app.db`). Plik `core.py` trzyma połączenie, wątkowo-bezpieczny `_execute_query` (chroniony przez `RLock`, bo połączenie jest współdzielone między wątkami przy `check_same_thread=False`) oraz `_initialize_db()`, które tworzy ~20 tabel (`CREATE TABLE IF NOT EXISTS`).
- Warstwa wykonuje też **lekkie migracje schematu** — sprawdza `PRAGMA table_info` i uruchamia `ALTER TABLE ADD COLUMN` dla starszych baz, by zaktualizować działającą bazę bez utraty danych użytkowników.

Dwa wbudowane serwery WWW

Bot to nie tylko klient Discorda — uruchamia też **serwer Flask w osobnym wątku**:

- `GET /` — endpoint uptime/health („Bot is alive and kicking!”).
- `POST /webhook/report/<token>` — webhook przychodzący, który pozwala zewnętrznej usłudze wepchnąć raport, a bot przekazuje go w DM do użytkownika. Endpoint jest zabezpieczony: weryfikacja podpisu **HMAC-SHA256**, **rate limiter** w oknie przesuwным, limit rozmiaru żądania i mapowanie `token`→użytkownik.

Most między wątkami realizuje `asyncio.run_coroutine_threadsafe`, bo Flask działa na innym wątku niż pętla zdarzeń asyncio bota.

Osobny plik `stock_proxy_service.py` to mała usługa Flask uruchamiana na **hoście**, która pobiera dane z Yahoo Finance i udostępnia je kontenerowi na porcie 9999 (przez `host.docker.internal`). Powstała, bo sieciowanie Dockera przeszkadzało w żądaniach do Yahoo Finance — to dobra „historia z życia” na rozmowę.

Konfiguracja — `config.py`

Używa **Pydantic Settings** (`BaseSettings`) do wczytania i **walidacji** zmiennych środowiskowych z `.env`. Pydantic daje typowaną, walidowaną konfigurację — jeśli brakuje wymaganego sekretu jak `DISCORD_BOT_TOKEN`, aplikacja zatrzymuje się od razu na starcie z czytelną informacją, zamiast paść później w niejasny sposób.

Integracje zewnętrzne — `api_clients/`

Każda usługa ma osobny moduł klienta: TMDb i TVMaze (seriale/filmy), Yahoo Finance i Alpha Vantage (giełda), OpenWeatherMap (pogoda), OpenLibrary (książki), Steam / PCGamingWiki / Wikipedia (gry), Google News RSS. Czysty podział: cogi obsługują logikę Discorda, klienci obsługują logikę HTTP/API.

Testy — `tests/`

`pytest` z `pytest-asyncio`. Plik `conftest.py` ustawia atrapy zmiennych środowiskowych (by walidacja config przeszła w CI) i dostarcza fixture `db_manager`, który tworzy **DataManager na tymczasowej bazie SQLite** dla każdego testu — warstwa danych jest testowana naprawdę, w izolacji, bez zależności zewnętrznych.

3. Biblioteki Python

Zależności aplikacji deklaruje plik `requirements.txt` (instalowany w obrazie Dockera). Poniżej jego pełna treść, a dalej omówienie roli każdej biblioteki.

```
discord.py
python-dotenv
requests>=2.28.0
Flask
yfinance>=0.2.18
urllib3>=1.26.0
google-generativeai
beautifulsoup4
pydantic
pydantic-settings
```

requirements.txt — pełna treść

Biblioteka	Rola w projekcie
<code>discord.py</code>	Framework bota Discord: połączenie z gateway (WebSocket), system komend, cogi, komendy slash/hybrydowe, obsługa zdarzeń i interakcji.
<code>python-dotenv</code>	Wczytywanie zmiennych środowiskowych z pliku <code>.env</code> do procesu (uzupełnia Pydantic Settings podczas lokalnego developmentu).
<code>requests</code>	Synchroniczny klient HTTP do wywołań zewnętrznych API (TMDB, OpenWeatherMap, Steam, Wikipedia itp.). Przypięta wersja <code>>=2.28.0</code> .
<code>Flask</code>	Lekki serwer WWW. Hostuje endpoint <code>health/uptime</code> , webhook raportów oraz osobną usługę <code>stock-proxy</code> .
<code>yfinance</code>	Pobieranie danych giełdowych z Yahoo Finance (notowania, szeregi czasowe). Wersja <code>>=0.2.18</code> .
<code>urllib3</code>	Niskopoziomowa warstwa HTTP (zależność <code>requests</code>); przypięta <code>>=1.26.0</code> dla zgodności/bezpieczeństwa.
<code>google-generativeai</code>	Oficjalne SDK Google Gemini — zasila coga asystenta AI.
<code>beautifulsoup4</code>	Parser HTML do scrapowania/wyciągania danych ze stron (np. PCGamingWiki, Wikipedia) tam, gdzie nie ma czystego API.
<code>pydantic</code>	Walidacja danych i modele typowane — baza dla <code>pydantic-settings</code> .
<code>pydantic-settings</code>	Typowana i walidowana konfiguracja z <code>.env</code> / zmiennych środowiskowych (klasa <code>Settings</code> w <code>config.py</code>).

W CI doinstalowywane są dodatkowo biblioteki testowe: `pytest` oraz `pytest-asyncio` (tryb `asyncio` w `pytest.ini` ustawiony na `auto`).

4. Dockerfile — analiza szczegółowa

Plik `bot.Dockerfile` buduje obraz aplikacji. To build **jednoetapowy** (single-stage). Poniżej pełna, niezmieniona zawartość pliku, a następnie omówienie każdego fragmentu wraz z uzasadnieniem decyzji.

```
# Use an official Python runtime as a parent image
FROM python:3.12-slim

# Install curl for debugging network issues
# Also install ca-certificates for HTTPS connections and dnsutils for DNS debugging
# Added build-essential for compiling python packages if wheels are missing
RUN apt-get update && apt-get install -y \
    curl \
    ca-certificates \
    dnsutils \
    iputils-ping \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container at /app
COPY requirements.txt .

# Install any needed packages specified in requirements.txt
# Upgrade pip first to ensure we can install newer wheels
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code into the container at /app
COPY bot.py .
COPY config.py .
COPY logger.py .
COPY data_manager.py .
COPY data_manager_impl/ ./data_manager_impl/
COPY cogs/ ./cogs/
COPY api_clients/ ./api_clients/
COPY utils/ ./utils/

# Expose port 5000 for the embedded Flask server
EXPOSE 5000

# Define the command to run the application
CMD ["python", "bot.py"]
```

bot.Dockerfile — pełna treść

Analiza linia po linii

Obraz bazowy

```
FROM python:3.12-slim
```

- Oficjalny obraz Pythona 3.12, wariant **-slim**.
- **slim** utrzymuje mały rozmiar obrazu (bez kompilatorów/dokumentacji), ale wciąż bazuje na Debianie, więc można doinstalować pakiety przez `apt-get`.
- Dlaczego nie Alpine? Alpine używa musl libc, co często psuje gotowe „wheels” Pythona i wymusza wolną kompilację ze źródeł. Dla Pythona **slim** to bezpieczniejszy domyślny wybór.

Pakiety systemowe

```
RUN apt-get update && apt-get install -y \
    curl ca-certificates dnsutils iputils-ping build-essential \
```

```
&& rm -rf /var/lib/apt/lists/*
```

- `curl`, `dnsutils`, `iputils-ping` — narzędzia do **debugowania sieci** (ten bot miał realne problemy z DNS/siecią).
- `ca-certificates` — certyfikaty do połączeń HTTPS.
- `build-essential` — kompilatory, by pip mógł **zbudować pakiet**, który nie ma gotowego wheela.
- **Kluczowy detal:** `rm -rf /var/lib/apt/lists/*` w tym samym RUN usuwa cache apt w tej samej warstwie, by nie powiększał obrazu. Gdyby sprzątanie było w osobnym RUN, cache zostałby w warstwie wcześniejszej.

Warstwa zależności — najważniejsza koncepcja

```
WORKDIR /app
COPY requirements.txt .
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt
```

- **Kopiujemy requirements.txt PRZED kodem aplikacji.** To najważniejsza koncepcja Dockera do omówienia: **cache warstw**.
- Docker cache'uje każdą warstwę. Dopóki `requirements.txt` się nie zmienia, kosztowna warstwa `pip install` jest używana ponownie nawet, gdy zmienisz tylko `bot.py`.
- Gdyby najpierw skopiować cały kod, każda zmiana w kodzie unieważniałaby cache zależności.
- `--no-cache-dir` trzyma cache pobierania pip poza obrazem (mniejszy obraz).

Kopiowanie kodu aplikacji

```
COPY bot.py .
COPY config.py .
COPY logger.py .
COPY data_manager.py .
COPY data_manager_impl/ ./data_manager_impl/
COPY cogs/ ./cogs/
COPY api_clients/ ./api_clients/
COPY utils/ ./utils/
```

- Kod kopiowany jest jawnie (zamiast `COPY . .`), co utrzymuje śmieci poza obrazem.
- Te warstwy zmieniają się często, dlatego są na końcu — by nie unieważniać warstwy zależności powyżej.

Port i komenda startowa

```
EXPOSE 5000
CMD ["python", "bot.py"]
```

- `EXPOSE 5000` dokumentuje port Flaska — to metadana, sam w sobie nie publikuje portu; robi to `-p 5000:5000` przy `docker run`.
- `CMD` w **formie exec** (tablica JSON), a nie formie shell. Forma exec sprawia, że proces jest bezpośrednio PID 1 i poprawnie odbiera sygnały (np. SIGTERM) do czystego zamknięcia.

Częste pytania o Dockerfile

Jak zmniejszyć / zabezpieczyć obraz? Build wieloetapowy (kompilacja zależności w etapie buildera, kopiowanie tylko zainstalowanych pakietów do czystego etapu runtime) oraz dodanie użytkownika nie-root (`USER`). Dobry punkt jako „co bym poprawił”.

Dlaczego nie Alpine? musli libc często łamie wheels Pythona i wymusza wolne kompilacje — slim jest bezpieczniejszy dla Pythona.

5. GitHub Actions — pipeline CI/CD

Plik `.github/workflows/deploy.yml` definiuje pełny pipeline. Poniżej jego kompletna treść, a następnie analiza wyzwalacza i trzech zadań (jobs).

```
name: Deploy to Oracle Cloud

on:
  push:
    branches: [ "main" ]
    workflow_dispatch:

env:
  REGISTRY: ghcr.io
  # IMAGE_NAME will be set dynamically to ensure lowercase

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          # Install app dependencies
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
          # Install testing dependencies
          pip install pytest pytest-asyncio

      - name: Run Tests
        run: |
          # Debug: List files
          ls -R
          # Add the bot directory to PYTHONPATH so tests can find the modules
          export PYTHONPATH=$PYTHONPATH:$(pwd)
          pytest -vv tests/

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Downcase REPO
        run: |
          echo "REPO_LC=${GITHUB_REPOSITORY,,}" >>${GITHUB_ENV}

      - name: Log in to the Container registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
```

```

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Extract metadata (tags, labels)
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ env.REGISTRY }}/${{ env.REPO_LC }}
    tags: |
      type=raw,value=latest
      type=sha

- name: Create dummy .env file
  run: |
    echo "DISCORD_BOT_TOKEN=dummy" >> .env
    echo "TMDB_API_KEY=dummy" >> .env
    echo "ALPHA_VANTAGE_API_KEY=dummy" >> .env
    echo "OPENWEATHERMAP_API_KEY=dummy" >> .env
    echo "GEMINI_API_KEY=dummy" >> .env

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    # Context must be the subdirectory where Dockerfile and code exist
    context: .
    file: bot.Dockerfile
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

deploy:

```

needs: build-and-push
runs-on: ubuntu-latest
steps:

```

```

- name: Downcase REPO
  run: |
    echo "REPO_LC=${{GITHUB_REPOSITORY,,}}" >>${{GITHUB_ENV}}

```

Avoid Docker-based ssh actions (they can fail building/pulling on runners).

```

- name: Set up SSH agent
  uses: webfactory/ssh-agent@v0.9.0
  with:
    ssh-private-key: ${{ secrets.OCI_SSH_KEY }}

```

```

- name: Add server to known_hosts
  run: |
    mkdir -p ~/.ssh
    ssh-keyscan -H "${{ secrets.OCI_HOST }}" >> ~/.ssh/known_hosts

```

```

- name: Create .env file from secret
  env:
    ENV_FILE_CONTENT: ${{ secrets.ENV_FILE }}
    FIREBASE_DATABASE_URL: ${{ secrets.FIREBASE_DATABASE_URL }}
    FIREBASE_DATABASE_SECRET: ${{ secrets.FIREBASE_DATABASE_SECRET }}
    TIMER_OWNER_ID: ${{ secrets.TIMER_OWNER_ID }}
    DISCORD_SYNC_SECRET: ${{ secrets.DISCORD_SYNC_SECRET }}
    TIMER_AUTH_PASSWORD: ${{ secrets.TIMER_AUTH_PASSWORD }}
  run: |
    # Preserve content exactly (including newlines)
    printf '%s' "$ENV_FILE_CONTENT" > .env
    # Drop any existing lines for keys we inject from repo secrets, so we never duplicate
    # (ENV_FILE often already contained FIREBASE_* from older edits – duplicate keys in
    # Docker --env-file are last-wins and are easy to misread in grep/debug).
    awk '
      /^FIREBASE_DATABASE_URL=/ { next }
      /^FIREBASE_DATABASE_SECRET=/ { next }
      /^TIMER_OWNER_ID=/ { next }
      /^DISCORD_SYNC_SECRET=/ { next }

```



```

    /^TIMER_AUTH_PASSWORD=/ { next }
    { print }
' .env > .env.tmp && mv .env.tmp .env
# Append from GitHub secrets (only non-empty – empty secret = do not inject that key)
[ -n "$FIREBASE_DATABASE_URL" ] && echo "FIREBASE_DATABASE_URL=$FIREBASE_DATABASE_URL" >> .env
[ -n "$FIREBASE_DATABASE_SECRET" ] && echo "FIREBASE_DATABASE_SECRET=$FIREBASE_DATABASE_SECRET" >> .env
[ -n "$TIMER_OWNER_ID" ] && echo "TIMER_OWNER_ID=$TIMER_OWNER_ID" >> .env
[ -n "$DISCORD_SYNC_SECRET" ] && echo "DISCORD_SYNC_SECRET=$DISCORD_SYNC_SECRET" >> .env
[ -n "$TIMER_AUTH_PASSWORD" ] && echo "TIMER_AUTH_PASSWORD=$TIMER_AUTH_PASSWORD" >> .env

- name: Copy .env to server
  run: |
    scp .env "${{ secrets.OCI_USERNAME }}@${{ secrets.OCI_HOST }}:~/ .env"

- name: Deploy over SSH (no action container)
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
    REGISTRY: ${{ env.REGISTRY }}
    IMAGE: ${{ env.REGISTRY }}/${{ env.REPO_LC }}:latest
    ACTOR: ${{ github.actor }}
  run: |
    # Pipe token via stdin so it never needs to exist as a remote env var.
    # REGISTRY/IMAGE/ACTOR expand locally; $HOME expands remotely.
    printf '%s' "$GITHUB_TOKEN" | ssh "${{ secrets.OCI_USERNAME }}@${{ secrets.OCI_HOST }}" "bash -lc 'set -euo pipefail
    docker login \"${REGISTRY}\" -u \"${ACTOR}\" --password-stdin
    docker pull \"${IMAGE}\"
    docker stop bot-container || true
    docker rm bot-container || true
    mkdir -p \"${HOME}/data\"
    docker run -d \
      --name bot-container \
      --restart unless-stopped \
      -v \"${HOME}/data:/app/data\" \
      --env-file \"${HOME}/.env\" \
      -p 5000:5000 \
      \"${IMAGE}\"
    docker image prune -f
    ,,"

```

.github/workflows/deploy.yml — pełna treść

Wyzwalacz

```

on:
  push:
    branches: [ "main" ]
  workflow_dispatch:

```

Uruchamia się przy każdym pushu do gałęzi `main`, a `workflow_dispatch` pozwala uruchomić go ręcznie z poziomu interfejsu GitHub.

Pipeline składa się z **trzech sekwencyjnych zadań** połączonych `needs`: — to właśnie bramkowanie (gating) jest najważniejsze.

Job 1 — test

```

test:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.12'
        cache: 'pip'

    - name: Install dependencies

```

```

run: |
  python -m pip install --upgrade pip
  # Install app dependencies
  if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
  # Install testing dependencies
  pip install pytest pytest-asyncio

- name: Run Tests
  run: |
    # Debug: List files
    ls -R
    # Add the bot directory to PYTHONPATH so tests can find the modules
    export PYTHONPATH=$PYTHONPATH:$(pwd)
    pytest -vv tests/

```

- Checkout kodu, ustawienie Pythona 3.12 (z cache pip między uruchomieniami).
- Instalacja `requirements.txt` + `pytest` i `pytest-asyncio`.
- Ustawienie `PYTHONPATH` na katalog główny repo, by testy widziały moduły.
- Uruchomienie `pytest -vv tests/`.
- **To bramka jakości:** jeśli testy padną, pipeline zatrzymuje się tu i nic nie zostaje zbudowane ani wdrożone.

Job 2 — build-and-push (needs: test)

Uruchamia się tylko, gdy testy przeszły.

```

build-and-push:
  needs: test
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Downcase REPO
      run: |
        echo "REPO_LC=${GITHUB_REPOSITORY,,}" >>${GITHUB_ENV}

    - name: Log in to the Container registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Extract metadata (tags, labels)
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ env.REGISTRY }/${ env.REPO_LC }
        tags: |
          type=raw,value=latest
          type=sha

    - name: Create dummy .env file
      run: |
        echo "DISCORD_BOT_TOKEN=dummy" >> .env
        echo "TMDB_API_KEY=dummy" >> .env
        echo "ALPHA_VANTAGE_API_KEY=dummy" >> .env
        echo "OPENWEATHERMAP_API_KEY=dummy" >> .env

```

```

    echo "GEMINI_API_KEY=dummy" >> .env

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    # Context must be the subdirectory where Dockerfile and code exist
    context: .
    file: bot.Dockerfile
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

- **permissions: packages: write** — nadaje wbudowanemu `GITHUB_TOKEN` prawo wypychania do GHCR.
- **Zmiana nazwy repo na małe litery:** `REPO_LC=${GITHUB_REPOSITORY,,}` — rejestry Dockera wymagają małych liter, a nazwy repo na GitHubie mogą mieć wielkie.
- Logowanie do **GHCR** (ghcr.io) przez `github.actor` i wbudowany `secrets.GITHUB_TOKEN` — bez ręcznie zarządzanego hasła do rejestru.
- **Buildx + metadata-action** generują tagi: `latest` oraz tag oparty o `sha` commita (każdy obraz jest identyfikowalny do commita).
- Build z `bot.Dockerfile`, push, oraz **cache GitHub Actions** (`cache-from/to: type=gha`) przyspieszający budowanie warstw między uruchomieniami.

Job 3 — deploy (needs: build-and-push)

To część robiąca wrażenie — pokazuje dyscyplinę w obsłudze sekretów i unika kruchych akcji firm trzecich na rzecz czystego SSH.

```

deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  steps:
    - name: Downcase REPO
      run: |
        echo "REPO_LC=${GITHUB_REPOSITORY,,}" >> ${GITHUB_ENV}

    # Avoid Docker-based ssh actions (they can fail building/pulling on runners).
    - name: Set up SSH agent
      uses: webfactory/ssh-agent@v0.9.0
      with:
        ssh-private-key: ${{ secrets.OCI_SSH_KEY }}

    - name: Add server to known_hosts
      run: |
        mkdir -p ~/.ssh
        ssh-keyscan -H "${secrets.OCI_HOST}" >> ~/.ssh/known_hosts

    - name: Create .env file from secret
      env:
        ENV_FILE_CONTENT: ${{ secrets.ENV_FILE }}
        FIREBASE_DATABASE_URL: ${{ secrets.FIREBASE_DATABASE_URL }}
        FIREBASE_DATABASE_SECRET: ${{ secrets.FIREBASE_DATABASE_SECRET }}
        TIMER_OWNER_ID: ${{ secrets.TIMER_OWNER_ID }}
        DISCORD_SYNC_SECRET: ${{ secrets.DISCORD_SYNC_SECRET }}
        TIMER_AUTH_PASSWORD: ${{ secrets.TIMER_AUTH_PASSWORD }}
      run: |
        # Preserve content exactly (including newlines)
        printf '%s' "$ENV_FILE_CONTENT" > .env
        # Drop any existing lines for keys we inject from repo secrets, so we never duplicate
        # (ENV_FILE often already contained FIREBASE_* from older edits – duplicate keys in
        # Docker --env-file are last-wins and are easy to misread in grep/debug).
        awk '
          /^FIREBASE_DATABASE_URL=/ { next }

```

```

/^FIREBASE_DATABASE_SECRET=/ { next }
/^TIMER_OWNER_ID=/ { next }
/^DISCORD_SYNC_SECRET=/ { next }
/^TIMER_AUTH_PASSWORD=/ { next }
{ print }
' .env > .env.tmp && mv .env.tmp .env
# Append from GitHub secrets (only non-empty — empty secret = do not inject that key)
[ -n "$FIREBASE_DATABASE_URL" ] && echo "FIREBASE_DATABASE_URL=$FIREBASE_DATABASE_URL" >> .env
[ -n "$FIREBASE_DATABASE_SECRET" ] && echo "FIREBASE_DATABASE_SECRET=$FIREBASE_DATABASE_SECRET" >> .env
[ -n "$TIMER_OWNER_ID" ] && echo "TIMER_OWNER_ID=$TIMER_OWNER_ID" >> .env
[ -n "$DISCORD_SYNC_SECRET" ] && echo "DISCORD_SYNC_SECRET=$DISCORD_SYNC_SECRET" >> .env
[ -n "$TIMER_AUTH_PASSWORD" ] && echo "TIMER_AUTH_PASSWORD=$TIMER_AUTH_PASSWORD" >> .env

- name: Copy .env to server
  run: |
    scp .env "${{ secrets.OCI_USERNAME }}@${{ secrets.OCI_HOST }}:~/ .env"

```

- [webfactory/ssh-agent](#) łączy **klucz prywatny SSH** (`secrets.OCI_SSH_KEY`) do agenta.
- [ssh-keyscan](#) dodaje serwer do `known_hosts`, by SSH nie pytało o autentyczność hosta.
- **Budowanie pliku .env z sekretów:** `printf` zapisuje zbiorczy sekret `ENV_FILE`, a następnie `awk` usuwa zduplikowane klucze przed ponownym dopisaniem konkretnych sekretów Firebase/timer. Powód: `docker --env-file` przy duplikatach stosuje regułę „ostatni wygrywa”, co łatwo źle odczytać podczas debugowania.
- `scp` kopiuje `.env` na serwer.

Ostatni krok — właściwe przewdrożenie kontenera na serwerze:

```

- name: Deploy over SSH (no action container)
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    REGISTRY: ${ env.REGISTRY }
    IMAGE: ${ env.REGISTRY }/${ env.REPO_LC }:latest
    ACTOR: ${ github.actor }
  run: |
    # Pipe token via stdin so it never needs to exist as a remote env var.
    # REGISTRY/IMAGE/ACTOR expand locally; $HOME expands remotely.
    printf '%s' "$GITHUB_TOKEN" | ssh "${{ secrets.OCI_USERNAME }}@${{ secrets.OCI_HOST }}" "bash -lc 'set -e pipefail
    docker login \"$REGISTRY\" -u \"$ACTOR\" --password-stdin
    docker pull \"$IMAGE\"
    docker stop bot-container || true
    docker rm bot-container || true
    mkdir -p \"$HOME/data\"
    docker run -d \
      --name bot-container \
      --restart unless-stopped \
      -v \"$HOME/data:/app/data\" \
      --env-file \"$HOME/.env\" \
      -p 5000:5000 \
      \"$IMAGE\"
    docker image prune -f
    '"

```

- Token rejestru jest podawany przez **stdin** (`printf ... | ssh ...`), więc nigdy nie łąduje w zmiennej środowiskowej ani w historii powłoki.
- `bash -lc 'set -e pipefail ...'` — tryb ścisły: skrypt pada głośno przy każdym błędzie.
- `docker stop bot-container || true` — `|| true` oznacza „nie przerywaj, jeśli kontener jeszcze nie istnieje”.
- `-v $HOME/data:/app/data` — bind-mount (wolumen), dzięki któremu baza SQLite **przeżywa przewdrożenie**. Bez tego każdy deploy kasowałby dane użytkowników — to najważniejszy detal produkcyjny.
- `--restart unless-stopped` — kontener wraca po restarcie/awarii serwera.
- `docker image prune -f` — sprząta stare „dangling” obrazy (odzyskuje miejsce).

- **Ekspansja zmiennych lokalnie vs zdalnie:** `$REGISTRY/$IMAGE` rozwijają się na runnerze; `\$HOME` jest eskejpowane, by rozwinąć się na serwerze. Wiedza, po której stronie rozwija się zmienna, to ostry detal.

Sekrety użyte w pipeline

Sekret	Zastosowanie
GITHUB_TOKEN	Wbudowany token — logowanie i push do GHCR (Job 2).
OCI_SSH_KEY	Klucz prywatny SSH do maszyny Oracle Cloud.
OCI_HOST / OCI_USERNAME	Adres i użytkownik serwera docelowego.
ENV_FILE	Zbiorcza zawartość pliku .env (sekrety runtime).
FIREBASE_DATABASE_URL / _SECRET	Dostęp do Firebase dla funkcji timer (owner-only).
TIMER_OWNER_ID, DISCORD_SYNC_SECRET, TIMER_AUTH_PASSWORD	Dodatkowe sekrety funkcji timer wstrzykiwane bez duplikatów.

Uwaga o niezgodności dokumentacji: plik `PIPELINE.md` opisuje deploy przez `docker compose up -d`, podczas gdy rzeczywisty `deploy.yml` używa surowego `docker run`. Jeśli padnie pytanie: dokumentacja opisuje wcześniejsze podejście oparte o `compose`; workflow obecnie robi przewdrożenie bezpośrednio przez `docker run`.

6. docker-compose.yml i Procfile

`docker-compose.yml` ułatwia lokalne uruchomienie (build + run jedną komendą) i definiuje politykę restartu oraz mapowanie portu.

```
services:
  discordbot:
    build:
      context: .
      dockerfile: bot.Dockerfile
    container_name: discordbot
    restart: unless-stopped
    environment:
      - PORT=5000
    ports:
      - "5000:5000"
```

docker-compose.yml

- `build.dockerfile: bot.Dockerfile` — wskazuje niestandardową nazwę Dockerfile.
- `restart: unless-stopped` — ta sama polityka co na produkcji.
- `ports: 5000:5000` — wystawia serwer Flask na hosta.

Procfile (format Heroku) deklaruje proces typu web:

web: python bot.py

Procfile

Pozostałość po platformie PaaS — mówi „uruchom proces web jako python bot.py”. Na obecnym wdrożeniu (Docker na Oracle Cloud) nie jest używany, ale warto wiedzieć, co to jest.

7. Komendy Linux / Docker do opanowania

Skoro wdrożenie idzie na maszynę Linux i budujesz obrazy Linuksowe, spodziewaj się pytań o linię komend. Pogrupowane tematycznie.

Docker

Komenda	Znaczenie
<code>docker build -f bot.Dockerfile -t discord-bot:latest</code>	Budowa obrazu z nazwanego Dockerfile.
<code>docker run -d --name bot-container --restart unless-stopped -p 5000:5000 -v \$HOME/data:/app/data --env-file .env</code>	Uruchomienie w tle: polityka restartu, mapowanie portu, wolumen, plik env.
<code>docker ps</code> / <code>docker ps -a</code>	Lista działających / wszystkich kontenerów.
<code>docker logs -f bot-container</code>	Śledzenie logów kontenera (debug na produkcji).
<code>docker exec -it bot-container bash</code>	Wejście do powłoki wewnątrz działającego kontenera.
<code>docker pull</code> / <code>stop</code> / <code>rm</code>	Pobranie, zatrzymanie, usunięcie kontenera.
<code>docker image prune -f</code>	Usunięcie „dangling” obrazów (odzysk miejsca).
<code>docker compose up -d</code> / <code>down</code>	Podniesienie / zatrzymanie stacku compose w tle.

SSH i transfer plików (używane w deployu)

- `ssh user@host, ssh-keygen, ssh-keyscan -H host >> ~/.ssh/known_hosts`
- `scp plik user@host:~/` — kopiowanie pliku na serwer.

Procesy i system (debug zawieszzonego bota)

- `ps aux | grep python, top / htop, kill -9 <pid>`
- `df -h` (dysk — obrazy/logi go zapełniają), `free -h` (pamięć)
- `systemctl status docker, journalctl -u docker`

Tekst i pliki (używane w samym workflow)

- `grep` (szukanie), `awk` (przetwarzanie pól/linii — deduplikacja kluczy env), `sed` (edycja strumieniowa)
- `cat, less, tail -f` (śledzenie logu), `chmod / chown` (uprawnienia — np. `chmod 600` na kluczu SSH)
- `printf '%s'` vs `echo` — `printf` zachowuje treść dokładnie (użyte do zapisu tokenu/.env bez zbędnych znaków nowej linii).

Koncepcje skryptowania powłoki w workflow

- `set -euo pipefail` — przerwij przy błędzie / niezdefiniowanej zmiennej / błędzie potoku.
- `cmd || true` — połącz nierozryw kod wyjścia, by skrypt szedł dalej.
- `>>` (dopisz) vs `>` (nadpisz); potok `|`; `--password-stdin` by trzymać sekrety poza linią komend.

8. Pytania rekrutacyjne — błyskawiczne odpowiedzi

Q: Dlaczego jeden bot z cogami zamiast mikroserwisów?

A: Jeden artefakt do wdrożenia, wspólne połączenie do DB i jedna pętla zdarzeń, niski koszt operacyjny na darmowej maszynie. Cogi i tak dają separację logiczną. Mikroserwisy dorzuciłyby koszt sieci/ops bez korzyści przy tej skali.

Q: Jak chronisz sekrety?

A: Nigdy nie commitowane (.env w .gitignore); sekrety CI w GitHub Secrets; sekrety runtime kopiowane przez scp na serwer; token rejestru podawany przez stdin; Pydantic waliduje je na starcie.

Q: Jak dane przeżywają deploy?

A: SQLite na bind-mount wolumenie (-v \$HOME/data:/app/data). Kontener jest jednorazowy, dane już nie.

Q: Co się dzieje, gdy test padnie?

A: Łańcuch needs: zatrzymuje pipeline na zadaniu test — brak builda, brak deployu.

Q: Dlaczego Flask wewnątrz bota Discord?

A: Endpoint health/uptime + odbieranie webhooków. Działa w wątku-demonie obok pętli asyncio, most przez run_coroutine_threadsafe.

Q: Największe wyzwanie?

A: Sieciowanie Dockera psuło wywołania do Yahoo Finance — rozwiązane usługą stock-proxy po stronie hosta; oraz niezawodne pojawianie się komend slash w DM (logika _enable_dm_for_app_commands).

Q: Co byś poprawił?

A: Build wieloetapowy + użytkownik nie-root; logowanie strukturalne/metryki; migracja z SQLite na Postgres przy większej współbieżności/skali; deploy bramkowany health-checkiem (weryfikacja nowego kontenera przed prune).

9. Streszczenie — wszystko w pigułce

Skondensowany przegląd do szybkiej powtórki przed rozmową. Po jednym zdaniu na obszar.

Obszar	W skrócie
Aplikacja	Bot na Discorda (Python, discord.py). Jeden proces, ~13 modułów-cogów (TV/filmy, giełda, pogoda, przypomnienia, nawyki, książki, gry, AI Gemini). Dane w SQLite przez DataManager złożony z miksów. Dodatkowo serwer Flask w wątku: health + webhook. Konfiguracja walidowana Pydantic.
Biblioteki	discord.py (framework), Flask (health/webhook), requests + yfinance (HTTP + giełda), google-genai (AI), beautifulsoup4 (scraping), pydantic + pydantic-settings (typowana config), python-dotenv. Testy: pytest + pytest-asyncio.
Dockerfile	Baza python:3.12-slim; narzędzia sieciowe + build-essential; kopiowanie requirements.txt PRZED kodem (cache warstw); pip install; kopia kodu; EXPOSE 5000; CMD w formie exec dla poprawnej obsługi sygnałów.
Workflow (CI/CD)	Push do main → (1) test: pytest jako bramka jakości; (2) build-and-push: build obrazu i push do GHCR z tagami latest + sha; (3) deploy: SSH na VM Oracle Cloud, zapis .env z sekretów, docker pull + odtworzenie kontenera z wolumenem danych i restart unless-stopped. Joby spięte needs:.
Linux/Docker	docker build/run/logs/exec/ps (kontenery), ssh/scp/ssh-keyscan (zdalny deploy), awk/grep/printf (przetwarzanie tekstu w workflow), set -euo pipefail (głośnie błędy), true (ignoruj oczekiwane błędy), -v host:kontener (trwały wolumen).

10. Problemy napotkane w projekcie

Te problemy są **udokumentowane w kodzie i historii commitów** — to wiarygodne, szczerze odpowiedzi na pytanie „z czym było najtrudniej”. Każdy ma ślad w repozytorium.

Problem	Ślad w repo	Rozwiązanie
Docker nie mógł połączyć się z Yahoo Finance	komentarz w nagłówku stock_proxy_service.py	Proxy Flask po stronie hosta; kontener woła je przez host.docker.internal.
Błędy rozwiązywania DNS w kontenerze	Dockerfile instaluje dnsutils/iputils-ping/curl; opcja BOT_DNS w skrypcie PS1	Narzędzia do debugowania + opcjonalna flaga --dns.
Komendy slash nie pojawiały się w DM	_enable_dm_for_app_commands w bot.py + commit „restore DM slash visibility”	Jawne ustawienie allowed_contexts + allowed_installs na każdej komendzie/grupie.
Opóźnienia/synchronizacja komend slash	logika sync w on_ready + komenda !sync	Kopia komend na poziom serwera (natychmiast) + ręczny !sync dla nowych serwerów.
Zduplikowane klucze w .env przy deployu	blok awk w deploy.yml + commit „dedupe Firebase/timer env keys”	Usunięcie wstrzykiwanych kluczy przed ponownym dopisaniem (reguła „ostatni wygrywa”).
Zmiany schematu działającej bazy bez utraty danych	PRAGMA table_info + ALTER TABLE w core.py	Addytywne migracje uruchamiane przy starcie.

Problem	Ślad w repo	Rozwiązanie
Wątek Flask vs pętla asyncio	RLock + asyncio.run_coroutine_threadsafe w warstwie danych	Bezpieczny most między wątkami; serializacja współdzielonego połączenia SQLite.
Ryzyko wycieku tokenu rejestru w deployu	printf ... ssh z --password-stdin	Sekret podawany przez stdin — nigdy w zmiennej środowiskowej ani historii powłoki.

11. Sugestie usprawnień

Twoja odpowiedź na „co byś zrobił dalej / co poprawił”. Pogrupowane tematycznie.

Docker

- Build **wieloletapowy** (builder + slim runtime) → mniejszy obraz, mniejsza powierzchnia ataku.
- Uruchamianie jako **użytkownik nie-root (USER)**; dodanie **HEALTHCHECK**.
- Przypięcie dokładnych wersji / plik lock (**pip-tools** / **uv**) → powtarzalne buildy.

Runtime / dane

- Zamiana serwera deweloperskiego Flask na **gunicorn/waitress** (dev server nie jest produkcyjny).
- Przejście z surowego **ALTER TABLE** na prawdziwe **migracje (Alembic)**; rozważenie **Postgres** przy większej współbieżności; użycie **aiosqlite**, by zapytania do bazy nie blokowały pętli zdarzeń.

CI/CD

- Dodanie **lintu + kontroli typów** (**ruff**, **mypy**) oraz wpięcie skanu **bandit** (masz już **bandit_report.json**) do pipeline'u.
- **Deploy bramkowany health-checkiem** z rollbackiem: weryfikacja zdrowia nowego kontenera przed **docker image prune**.
- **Menedżer sekretów** (lub przynajmniej **chmod 600** i ograniczony zakres) zamiast jawnego **.env** na maszynie.

Obserwowalność

- Logowanie strukturalne + rotacja logów + zapis logów na wolumen (obecnie logi trafiają tylko na stdout).

Dokument wygenerowany automatycznie na podstawie kodu źródłowego repozytorium tv_and_stockmarket_bot. Kod w listingach jest wierny 1:1 z plikami w repo.